

TASK 11:

12 Test Cases:

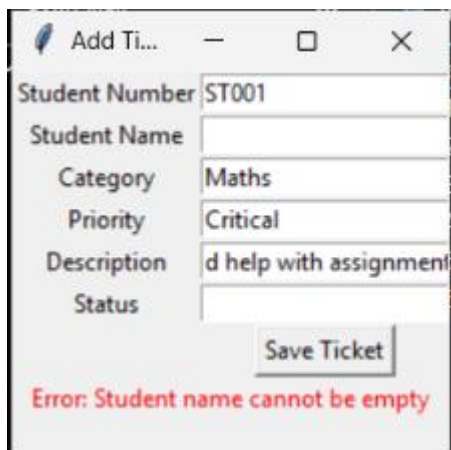
Test Case 1:

Validate that student number starts with “ST” and all fields are not empty and has been passed successfully.

```
def save_ticket(student_number, student_name, category, priority, description, status):  
    # Validation rules  
    if not student_number.startswith("ST"):  
        error_label.config(text="Error: Student number must start with 'ST'")  
        return  
    if len(description) < 10 or len(description) > 200:  
        error_label.config(text="Error: Description must be between 10 and 200 characters")  
        return  
    if not student_name.strip():  
        error_label.config(text="Error: Student name cannot be empty")  
        return
```

Test Case 2:

Student name cannot be empty and displays an error if empty.



The screenshot shows a window titled 'Add Ticket' with several input fields: 'Student Number' (containing 'ST001'), 'Student Name' (empty), 'Category' (containing 'Maths'), 'Priority' (containing 'Critical'), 'Description' (containing 'd help with assignment'), and 'Status' (empty). A 'Save Ticket' button is at the bottom. A red error message at the bottom of the window reads: 'Error: Student name cannot be empty'.

Test Case 3:

No <script> invalid input are allowed when adding a ticket and is passed successfully.

```

def save_ticket(student_number, student_name, category, priority, description, status):
    # Validation rules
    if not student_number.startswith("ST"):
        error_label.config(text="Error: Student number must start with 'ST'")
        return
    if len(description) < 10 or len(description) > 200:
        error_label.config(text="Error: Description must be between 10 and 200 characters")
        return
    if not student_name.strip():
        error_label.config(text="Error: Student name cannot be empty")
        return

    # Check all fields for unsafe input
    fields = {
        "Student Number": student_number,
        "Student Name": student_name,
        "Category": category,
        "Priority": priority,
        "Description": description,
        "Status": status
    }
    for field_name, field_value in fields.items():
        if not field_value.strip():
            error_label.config(text=f"Error: {field_name} is required")
            return
        if not is_safe_input(field_value):
            error_label.config(text=f"Error: Unsafe input detected in {field_name}")
            return

```

Test Case 4:

All tickets are displayed and available to view and is passed successfully.

ticket_id	student_number	student_name	category	description	priority	status	date_logged	closing_comment
T002	ST002	Jacobus Donald	Development	I have a problem acco	Critical		2026-06-12	High
T003	ST003	Sean Coetzee	English	im not very smart	Low		2026-06-13	Closed
T004	ST004	Tanija Coetzee	Afrikaans	i need urgent help wi	Critical		2026-06-13	Open
T005	ST005	Zack Coetzee	Accounting	My accounting apps	Critical		2026-06-13	Pending
T006	ST006	Jacob Zuma	Economics	Not alot of money to	Meduim		2026-06-13	Closed
T007	ST007	Ethan Rodger	LO	Broke my legs	Critical		2026-06-13	Pending
T008	ST008	Johann Huysamen	Management	Laptop Broke this mo	High		2026-06-13	
T009	ST009	Stephanie Groenewal	Software developmer	My computer crashes	Critical		2026-06-13	Open
T010	ST0010	Divan Basson	Software developmer	Power is down cant u	Critical		2026-06-13	
T011	ST001	Marco de Kok	Maths	Testing validations	Low	Open	2026-06-13	

Test Case 5:

Tickets can be searched by valid student ID and if wrong input is error like student name for example an

View & Manage Tickets								
ticket_id	student_number	student_name	category	description	priority	status	date_logged	closing_comment
T011	ST001	Marco de Kok	Maths	Testing validations	Low	Open	2026-06-13	

ST001

Search

error will be displayed.

View & Manage Tickets								
ticket_id	student_number	student_name	category	description	priority	status	date_logged	closing_comment

Marco Coetzee

Search

Filter by Status

Filter by Priority

Sort by Date

Info

No tickets found.

OK

Test Case 6:

You have the option to filter by status and priority when searching for ticket.

View & Manage Tickets								
ticket_id	student_number	student_name	category	description	priority	status	date_logged	closing_comment
T002	ST002	Jacobus Donald	Development	I have a problem acc	Critical		2026-06-12	High
T007	ST007	Ethan Rodger	LO	Broke my legs	Critical		2026-06-13	Pending

Search

Open

Filter by Status

Critical

Filter by Priority

Test Case 7:

The option to sort by date and priority should be available and working. Which it does.

ticket_id	student_number	student_name	category	description	priority	status	date_logged	closing_comment
T004	ST004	Tanija Coetzee	Afrikaans	i need urgent help wi	Critical		2026-06-13	Open
T005	ST005	Zack Coetzee	Accounting	My accounting apps	Critical		2026-06-13	Pending
T009	ST009	Stephanie Groenewal	Software developmer	My computer crashes	Critical		2026-06-13	Open
T010	ST0010	Divan Basson	Software developmer	Power is down cant u	Critical		2026-06-13	
T002	ST002	Jacobus Donald	Development	I have a problem acc	Critical		2026-06-12	High
T007	ST007	Ethan Rodger	LO	Broke my legs	Critical		2026-06-13	Pending
T008	ST008	Johann Huysamen	Management	Laptop Broke this mo	High		2026-06-13	
T003	ST003	Sean Coetzee	English	im not very smart	Low		2026-06-13	Closed
T011	ST001	Marco de Kok	Maths	Testing validations	Low	Open	2026-06-13	
T006	ST006	Jacob Zuma	Economics	Not alot of money to	Meduim		2026-06-13	Closed

Test Case 8:

Update existing tickets to new values and it is passed successfully.

ticket_id	student_number	student_name	category	description	priority	status	date_logged	closing_comment
T004	ST004	Tanija Coetzee	Afrikaans	i need urgent help wi	Critical		2026-06-13	Open
T005	ST005	Zack Coetzee	Accounting	My accounting apps	Critical		2026-06-13	Pending
T009	ST009	Stephanie Groenewal	Software developmer	My computer crashes	Critical		2026-06-13	Open
T010	ST0010	Divan Basson	Software developmer	Power is down cant u	Critical		2026-06-13	
T002	ST002	Jacobus Donald	Development	I have a problem acc	Critical		2026-06-12	High
T007	ST007	Ethan Rodger	LO	Broke my legs	Critical		2026-06-13	Pending
T008	ST008	Johann Huysamen	Management	Laptop Broke this mo	High		2026-06-13	
T003	ST003	Sean Coetzee	English	im not very smart	Low		2026-06-13	Closed
T011	ST001	Marco de Kok	Maths	Testing validations	Low	Open	2026-06-13	
T006	ST006	Jacob Zuma	Economics	Not alot of money to	Meduim		2026-06-13	Closed

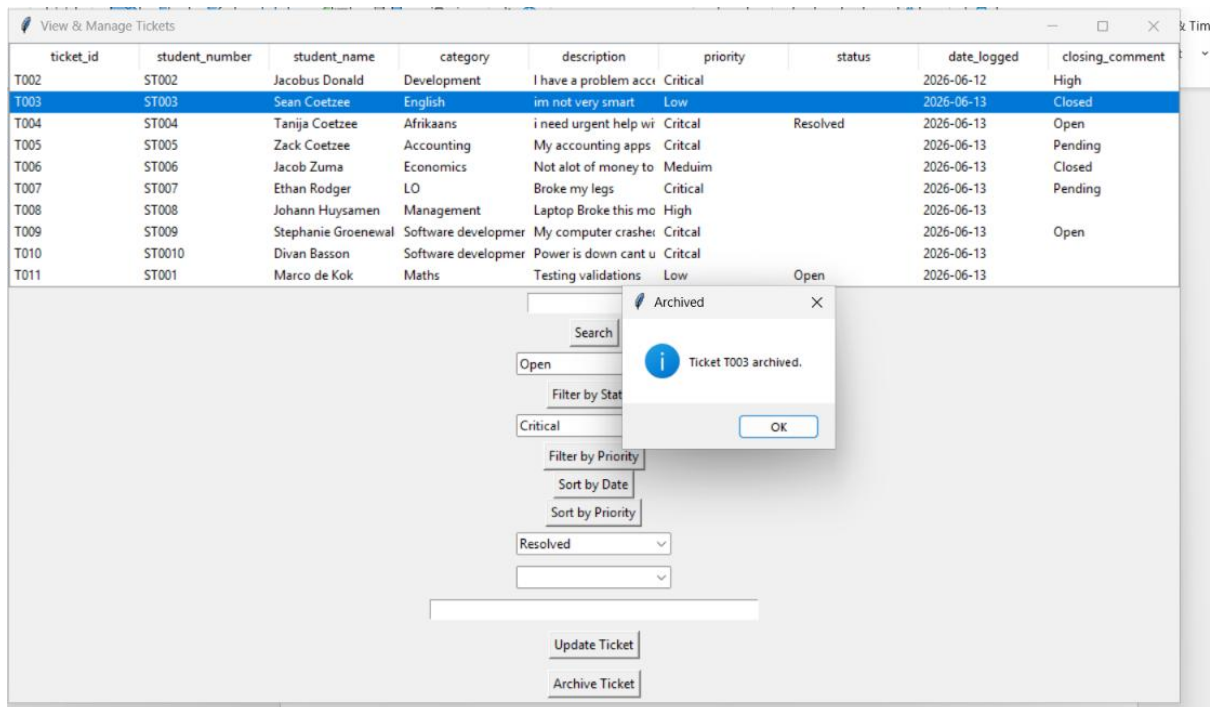
Success

Ticket T004 updated.

OK

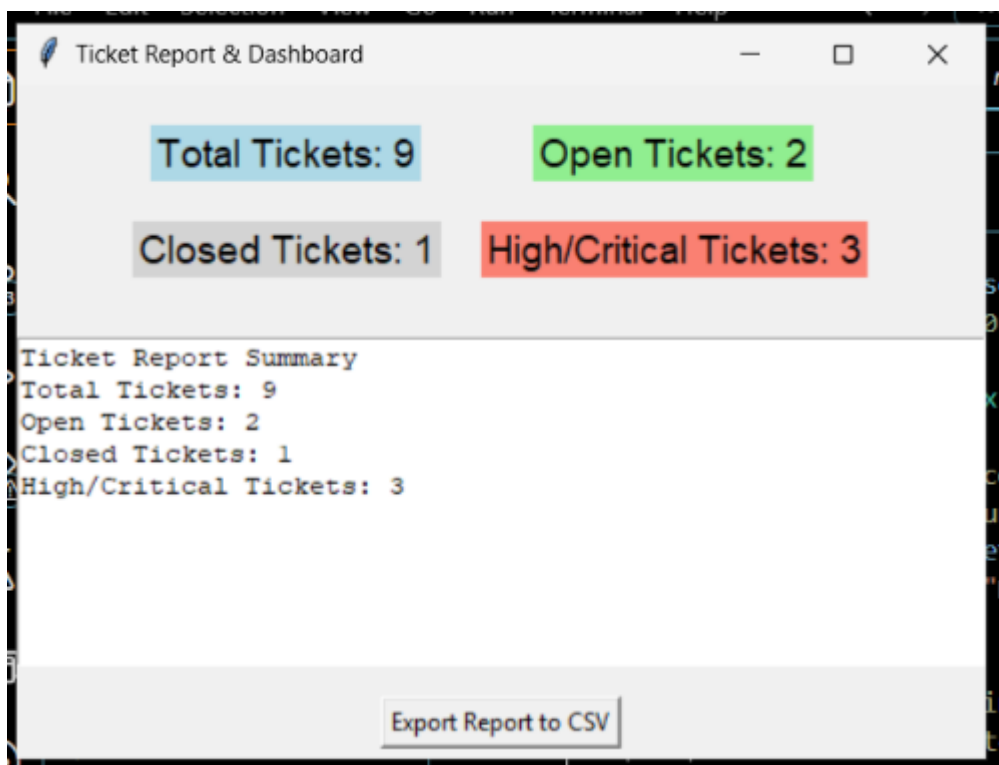
Test Case 9:

Archiving tickets should be possible, and it is possible.



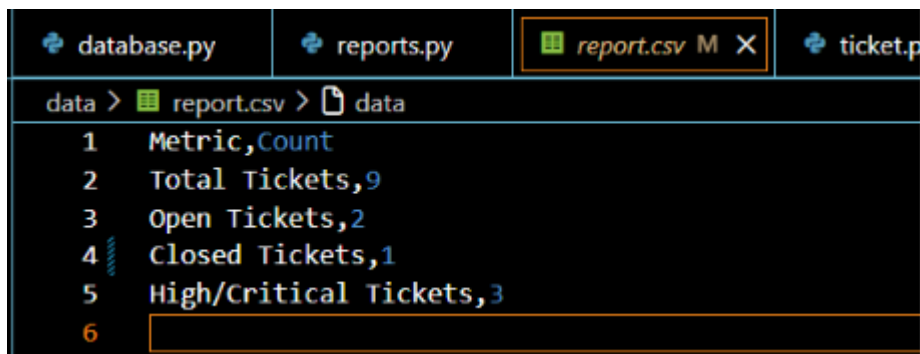
Test Case 10:

Dashboard should be displayed which displays the counts of all tickets and it is being displayed successfully.



Test Case 11:

You can export the report to CSV which is possible and working.

A screenshot of a code editor with four tabs: 'database.py', 'reports.py', 'report.csv M X', and 'ticket.p'. The 'report.csv' tab is active, showing a CSV file with the following content:

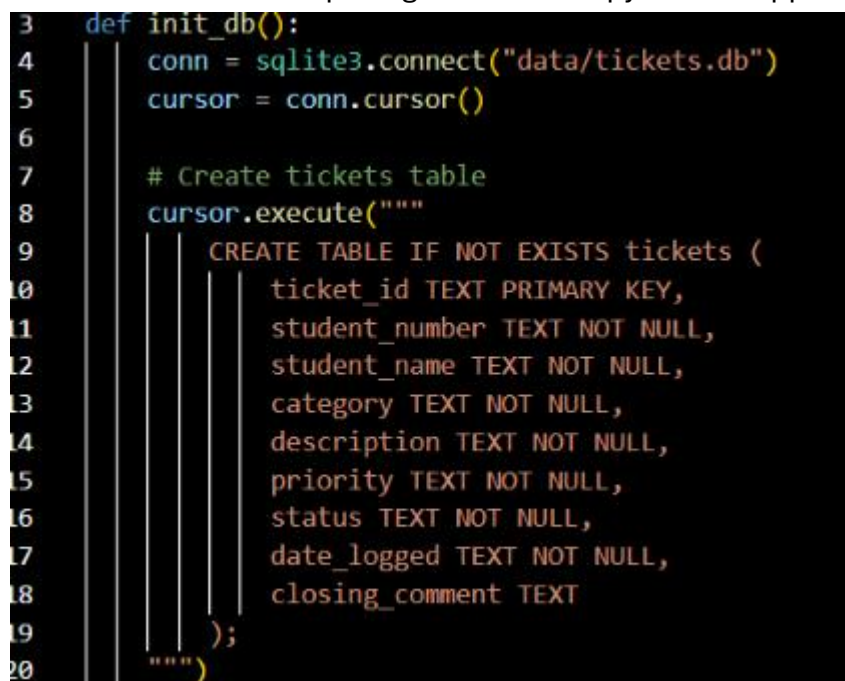
```
data > report.csv > data
1 Metric,Count
2 Total Tickets,9
3 Open Tickets,2
4 Closed Tickets,1
5 High/Critical Tickets,3
6
```

Test Case 12:

No personal information is displayed in CSV which is does not, only counts are displayed.

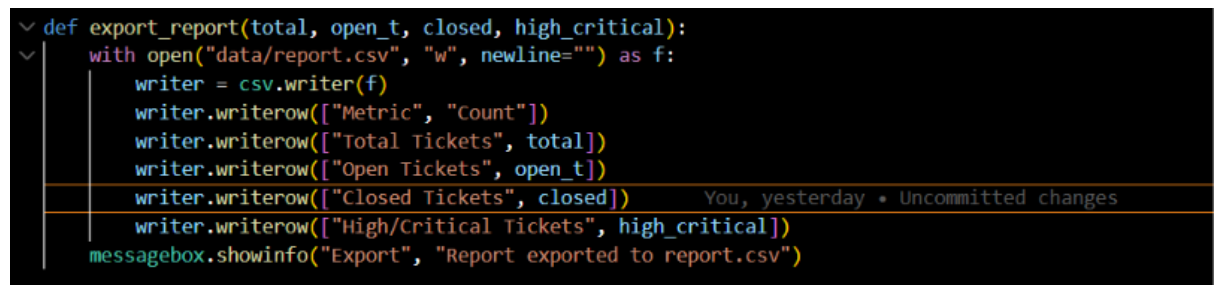
3 Debuggings:

1. I had a spelling error in tickets table I spelt category as category so the tickets did not save so I fixed the spelling error and ran python -m app.database again

A screenshot of Python code in a code editor, showing the initialization of a SQLite database. The code is as follows:

```
3 def init_db():
4     conn = sqlite3.connect("data/tickets.db")
5     cursor = conn.cursor()
6
7     # Create tickets table
8     cursor.execute("""
9         CREATE TABLE IF NOT EXISTS tickets (
10             ticket_id TEXT PRIMARY KEY,
11             student_number TEXT NOT NULL,
12             student_name TEXT NOT NULL,
13             category TEXT NOT NULL,
14             description TEXT NOT NULL,
15             priority TEXT NOT NULL,
16             status TEXT NOT NULL,
17             date_logged TEXT NOT NULL,
18             closing_comment TEXT
19         );
20     """)
```

2. My report.csv saved in the root of the project which is not good so I moved the location in saved to inside the app in the data folder.

A screenshot of Python code in a code editor, showing the export_report function. The code is as follows:

```
<v> def export_report(total, open_t, closed, high_critical):
<v>     with open("data/report.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Metric", "Count"])
        writer.writerow(["Total Tickets", total])
        writer.writerow(["Open Tickets", open_t])
        writer.writerow(["Closed Tickets", closed])
        writer.writerow(["High/Critical Tickets", high_critical])
        messagebox.showinfo("Export", "Report exported to report.csv")
```

3. I had an spelling error in CREATE TABLE IF NOT EXITS which corrected to EXISTS

```
3 def init_db():
4     conn = sqlite3.connect("data/tickets.db")
5     cursor = conn.cursor()
6
7     # Create tickets table
8     cursor.execute("""
9         CREATE TABLE IF NOT EXISTS tickets (
10             ticket_id TEXT PRIMARY KEY,
11             student_number TEXT NOT NULL,
12             student_name TEXT NOT NULL,
13             category TEXT NOT NULL,
14             description TEXT NOT NULL,
15             priority TEXT NOT NULL,
16             status TEXT NOT NULL,
17             date_logged TEXT NOT NULL,
18             closing_comment TEXT
19         );
20     """)
```